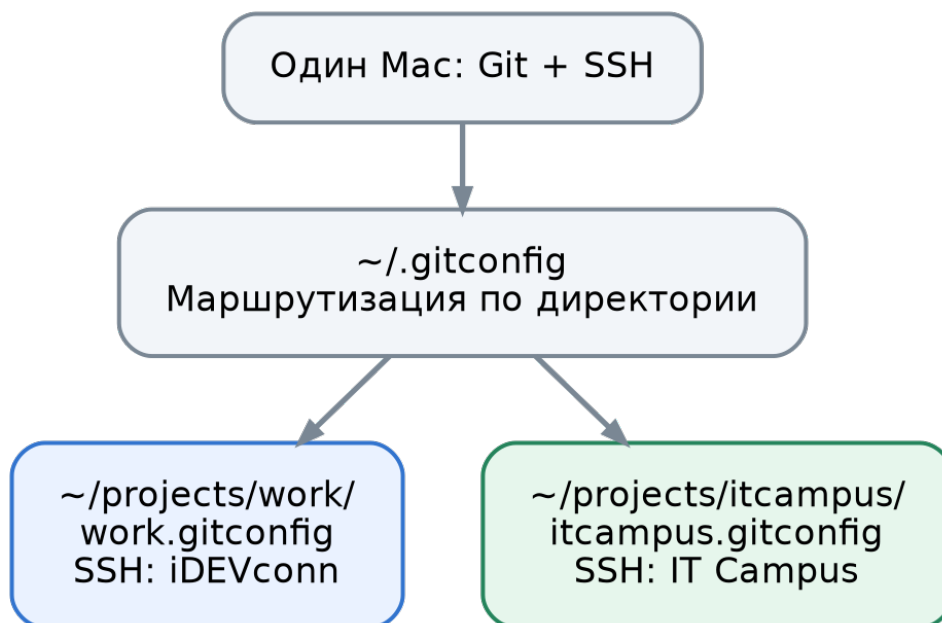


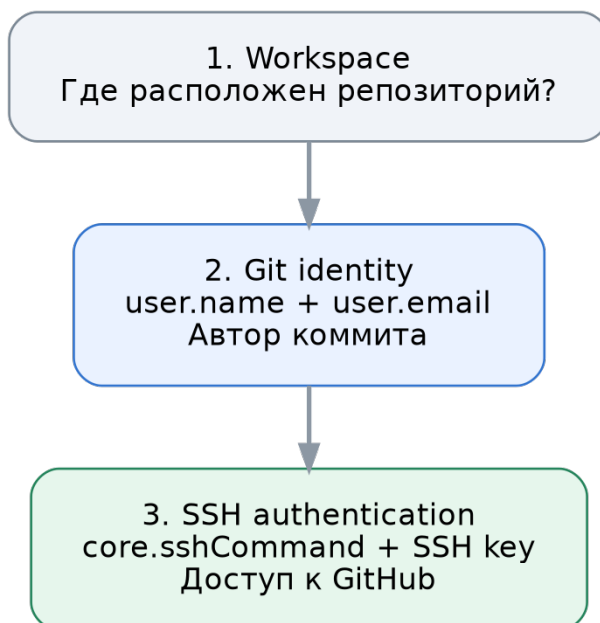
GitHub Workspaces

Как работать с несколькими GitHub-аккаунтами на одном Mac: отдельные SSH-ключи, Git identity и автоматический выбор конфигурации по расположению репозитория.



Общая архитектура: центральный маршрутизатор и два независимых workspace.

1. Что разделяем



Три уровня: расположение проекта, автор коммита и SSH-аутентификация.

Workspace выбирает конфигурацию через includelf. Git identity определяет автора коммита, но не права доступа. SSH authentication определяет ключ, который проверяет GitHub. Это логическое, а не системное разделение.

2. Структура каталогов

~/

```
.gitconfig
git-config/
  work.gitconfig
  itcampus.gitconfig
.ssh/
  config
  id_ed25519_iddevconn (+ .pub)
  github_itcampusdesc (+ .pub)
projects/
  work/
  itcampus/
```

```
mkdir -p ~/git-config ~/projects/work ~/projects/itcampus
```

3. SSH-ключи

Сначала проверьте существующие ключи. Не перезаписывайте приватные ключи. Создавайте новые только при необходимости.

```
ls -la ~/.ssh/
cat ~/.ssh/config
```

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_iddevconn \
-C "YOUR_PERSONAL_EMAIL"
ssh-keygen -t ed25519 -f ~/.ssh/github_itcampusdesc \
-C "YOUR_WORK_EMAIL"
```

Зарегистрируйте соответствующие публичные ключи в разных GitHub-аккаунтах: <https://github.com/settings/keys>. На macOS публичный ключ можно скопировать через pbcopy.

```
pbcopy < ~/.ssh/id_ed25519_iddevconn.pub
pbcopy < ~/.ssh/github_itcampusdesc.pub
```

4. SSH-алиасы: ~/.ssh/config

```
Host idevconn
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_iddevconn
  IdentitiesOnly yes
  AddKeysToAgent yes
  UseKeychain yes

Host github-work
  HostName github.com
  User git
  IdentityFile ~/.ssh/github_itcampusdesc
  IdentitiesOnly yes
  AddKeysToAgent yes
  UseKeychain yes
```

Host - локальный псевдоним, HostName - сервер GitHub, IdentityFile - выбранный SSH-ключ. Не создавайте дублирующие Host в существующем файле.

```
ssh -T git@idevconn
ssh -T git@github-work
```

5. Центральный маршрутизатор: ~/.gitconfig

Добавьте правила в существующую глобальную конфигурацию. Завершающий слэш позволяет сопоставлять вложенные репозитории.

```
[includeIf "gitdir:~/projects/work/"]
  path = ~/git-config/work.gitconfig

[includeIf "gitdir:~/projects/itcampus/"]
  path = ~/git-config/itcampus.gitconfig
```

6. Конфигурации аккаунтов

~/git-config/work.gitconfig

```
[user]
  name = Denis
  email = YOUR_PERSONAL_EMAIL

[core]
  sshCommand = ssh -i ~/.ssh/id_ed25519_idevconn -o IdentitiesOnly=yes
```

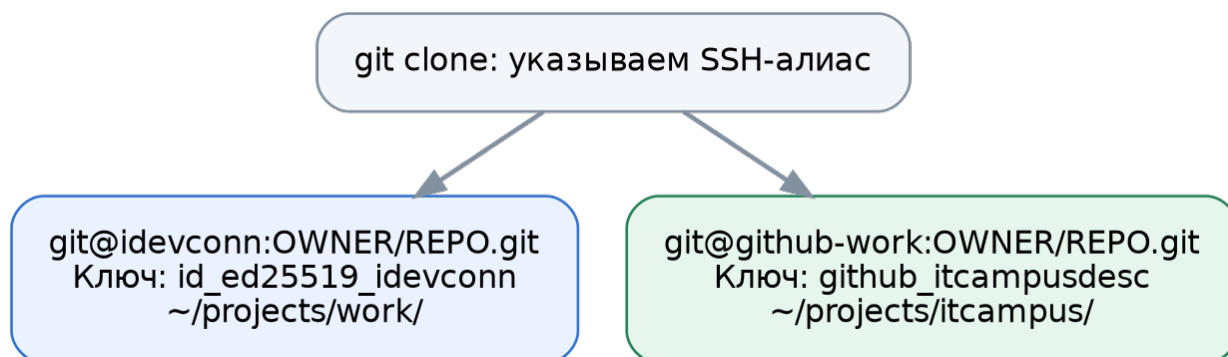
~/git-config/itcampus.gitconfig

```
[user]
  name = Denis
  email = YOUR_WORK_EMAIL

[core]
  sshCommand = ssh -i ~/.ssh/github_itcampusdesc -o IdentitiesOnly=yes
```

Замените email и имя на реальные значения. Локальный .git/config или GIT_SSH_COMMAND могут переопределить эти настройки.

7. Clone: используем SSH-алиасы



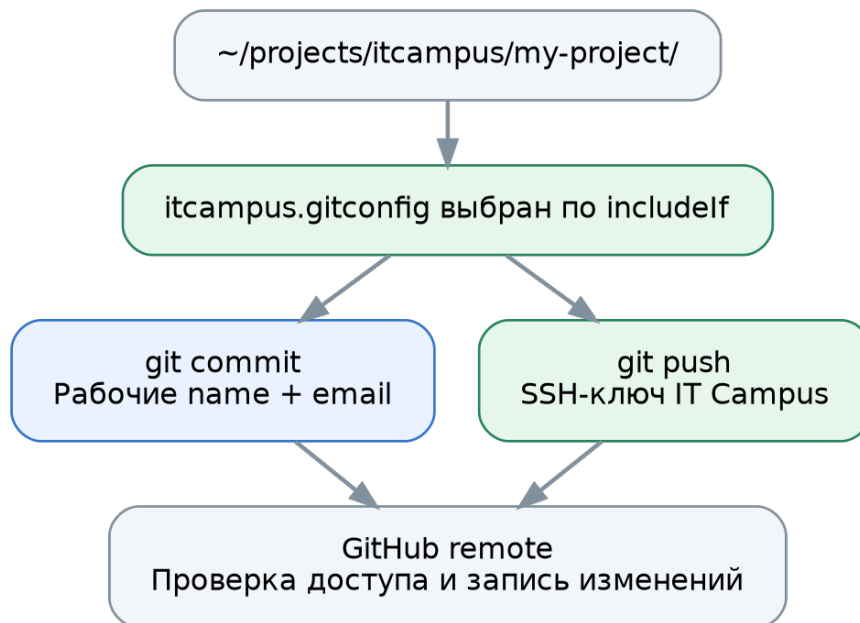
При первом клонировании ключ выбирается по SSH-алиасу в URL.

Во время первого clone будущего репозитория ещё нет, поэтому условная Git-конфигурация его директории не гарантирована. Укажите SSH-алиас прямо в URL.

```
cd ~/projects/work
git clone git@idevconn:idevconn/iCore.git

cd ~/projects/itcampus
git clone git@github-work:itcampus/REPO.git
```

8. Commit и Push



После clone настройки выбираются автоматически внутри репозитория.

```

cd ~/projects/itcampus/REPO
git add .
git commit -m "Update project"
git push

```

Git identity отвечает за авторство коммита, core.sshCommand - за SSH-ключ. Адрес отправки определяется remote; GitHub дополнительно проверяет права аккаунта.

9. Проверка и диагностика

```

git config --show-origin --get user.name
git config --show-origin --get user.email
git config --show-origin --get core.sshCommand
git remote -v
git config --show-origin --show-scope --list

```

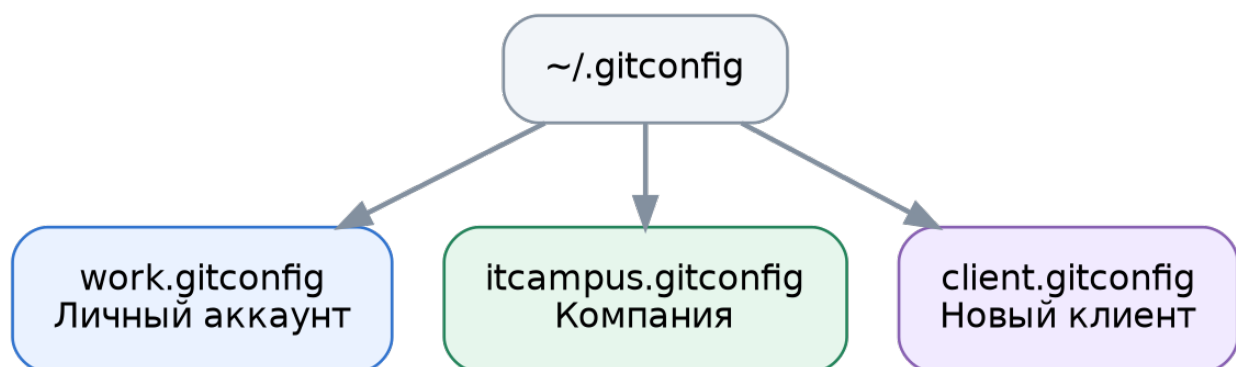
```

ssh -G github-work | grep -E '^(hostname|user|identityfile) '
ssh -T git@github-work

```

Если ключ выбран неожиданно, проверьте .git/config, переменную GIT_SSH_COMMAND и SSH-адрес remote.

10. Добавление третьего аккаунта



Каждый новый аккаунт получает собственные ключ, конфигурацию и директорию.

Создайте SSH-ключ, зарегистрируйте публичную часть, добавьте Host client в ~/.ssh/config и файл ~/git-config/client.gitconfig. Затем добавьте новое правило:

```
[includeIf "gitdir:~/projects/client/"]
  path = ~/git-config/client.gitconfig
```

Создайте ~/projects/client/ и клонируйте репозитории через git@client:OWNER/REPO.git. Существующие настройки других аккаунтов менять не нужно.

Сценарий для видеодемонстрации

Откройте два терминала с репозиториями из разных workspace. В каждом покажите user.email и core.sshCommand через git config --show-origin, затем проверьте SSH-алиасы через ssh -T и выполните Push в два разных репозитория.

Справка: git-scm.com/docs/git-config#_conditional_includes;
docs.github.com/en/authentication/connecting-to-github-with-ssh